Double machine learning for estimating nonlinear treatment effects

Ethan Young (105391559)

STATS 201B

1 Introduction

Consider the partially linear regression (PLR) model [3] for a single observation:

$$Y = D\theta_0 + g_0(X) + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X, D] = 0,$$

where $Y \in \mathbb{R}$ is the response, $D \in \mathbb{R}$ is the treatment, $X \in \mathbb{R}^p$ is the vector of covariates, $\theta_0 \in \mathbb{R}$ is the treatment effect, $g_0(\cdot) :\in \mathbb{R}^p \to \mathbb{R}$ is the nuisance function, and $\varepsilon \in \mathbb{R}$ is the noise. We wish to estimate θ_0. A popular framework for estimating treatment effects in high-dimensional settings is double machine learning (DML) [2], which uses machine learning (ML) methods to estimate nuisance functions while preserving valid statistical inference for the target parameter. In this report, we (1) compare the performance of the DML estimator with ordinary least squares (OLS) and (2) extend the DML framework to estimate nonlinear treatment effect functions $f_0(D)$ that are approximated by a finite Hermite expansion.

2 Comparison with OLS

We first investigate the effectiveness of DML by designing a simulation study and studying how the choice of machine learning method used to estimate the nuisance function affects the estimation accuracy of θ_0. The synthetic data consists of 1000 total replications, $n = 500$ observations per dataset, and a true causal parameter of $\theta_0 = 2$. We vary the number of covariates (i.e., $p \in \{20, 100, 400\}$) to see how it affects the performance of the DML estimator in this high-dimensional setting. We generate data from a PRL model used in [2] with the `make_plr_CCDDHNR2018` function in the `DoubleML` R package. For simplicity we use the default arguments in our experiments. The ML methods we use are lasso regression and random forest (RF). The lasso model searches for the optimal regularization value using 5-fold cross validation. The RF is configured with 500 trees and a maximum depth of 5. The main loop runs for 10 iterations and tests three estimation methods for each dataset: (1) standard OLS, (2) DML with lasso, and (3) DML with RF. The OLS implementation runs a standard linear regression predicting the outcome using the treatment and all other covariates. We compute the mean estimate $\hat{\theta}$, bias, root mean squared error (RMSE), and the empirical coverage probability of the 95% confidence interval (CI).

p	**Method**	$\hat{\theta}$	**Bias**	**RMSE**	**Coverage**
20	OLS	1.99	-0.0140	0.0437	1
	DML (lasso)	1.99	-0.0110	0.0447	0.9
	DML (RF)	2.02	0.0200	0.0425	1
100	OLS	2.00	-0.00499	0.0464	1
	DML (lasso)	1.99	-0.00913	0.0385	1
	DML (RF)	2.07	0.0695	0.0801	0.7
400	OLS	2.01	0.0114	0.0707	1
	DML (lasso)	1.99	-0.00552	0.0445	1
	DML (RF)	2.10	0.103	0.106	0.1

Table 1: Simulation study results. The method yielding the smallest RMSE is highlighted.

Table 1 shows our results. We observe that DML with lasso performs better as p increases. Although OLS performs similarly well, we expect worse results for even larger n and p. We also see that the bias and RMSE increase as p increases when we run DML with RF. This is a case where more careful hyperparameter tuning can improve our results. The code, which contains all simulation details, is found in Appendix C.1.

3 Nonlinear treatment effects

Consider a more general treatment effect function $f_0(D)$ when $f_0(D)$ is not linear in D. One way we can deal with this is to approximate $f_0(D)$ using a K-term basis $\psi(D) = [\psi_1(D), ..., \psi_K(D)]^\top$, i.e.,

$$Y = \sum_{j=1}^{K} \beta_j \psi_j(D) + g_0(X) + \varepsilon$$

for a single observation. Here, $\beta_1, ..., \beta_K$ are the coefficients of the basis functions $\psi_1, ..., \psi_K$, respectively. To estimate the coefficients $\boldsymbol{\beta} = (\beta_1, ..., \beta_K)^\top$, we use the following procedure. We first use a ML model to Y from X in order to remove the part of the outcome variation that can be explained by the covariates X. Then, we predict each basis function $\psi_j(D)$ in the expansion using X. This orthogonalizes the treatment components with respect to the controls and ensures that any correlation between D and X is removed. Finally, we run OLS regression of the outcome residuals on the basis residuals. The resulting function is the estimated nonlinear treatment effect.

3.1 Hermite expansion

While there are a number of basis functions (e.g., splines, Fourier series), we want one that has particularly useful analytical and numerical properties. To that end, we consider an infinite expansion of the treatment effect function using the probabilist's Hermite polynomials:

$$f_0(x) = \sum_{k=0}^{\infty} c_k \mathrm{He}_k(x), \quad c_k = \frac{1}{k!} \int_{-\infty}^{\infty} f(x) \mathrm{He}_k(x) \frac{e^{-x^2/2}}{\sqrt{2\pi}} dx.$$

Using this basis expansion allows us to leverage the theory of orthogonal polynomials [4]. In particular, they are (in general) efficiently computed, numerically stable, and reduce approximation errors. Moreover, if the support of the input is a (standard) Gaussian, then the Hermite polynomials are orthogonal with respect to the Gaussian weight function. For brevity, all references to Hermite polynomials are henceforth assumed to mean the probabilist's Hermite polynomials.

Let us state the DML problem for a single observation more formally. Assume that $D \sim \mathcal{N}(0, 1)$. This ensures the polynomials satisfy the orthogonality condition. Let $f_0^{(K)}(D)$ denote the K-term Hermite expansion of $f_0(D)$, i.e.,

$$f_0^{(K)}(D) = \sum_{j=0}^{K} \beta_j \mathrm{He}_n(D).$$

We first generate the Hermite basis $\{\mathrm{He}_1(D), ..., \mathrm{He}_K(D)\}$. For every $\mathrm{He}_k(D)$, we train a ML model (e.g., lasso regression or RF) to find $\hat{\mathbb{E}}[\mathrm{He}_k(D) \mid X]$. We then compute the residuals:

$$\tilde{\mathrm{He}}_k(D) = \mathrm{He}_k(D) - \hat{\mathbb{E}}[\mathrm{He}_k(D) \mid X].$$

Finally, we regress the outcome residuals $\tilde{Y}$ onto the set of Hermite residuals:

$$\tilde{Y} = \sum_{j=1}^{K} \beta_j \tilde{\mathrm{He}}_j(D) + \varepsilon.$$

It is clear that the coefficients β_j estimated here are the Hermite coefficients. The key advantage of using Hermite polynomials instead of, say, standard polynomials is orthogonality: the cross-products of different Hermite terms integrate to zero: $\mathbb{E}[\mathrm{He}_m(D) \cdot \mathrm{He}_n(D)] = 0$ for $m \neq n$, which implies that the covariance matrix of the regressors ($\tilde{\mathrm{He}}$) will be approximately diagonal. This in turn minimizes the variance and stabilizes the coefficients β.

A relevant concern at this point is whether the finite expansion $f_0^{(K)}(D)$ a sufficiently good approximation of $f_0(D)$. For analytic functions, Hermite coefficients typically decay exponentially [1]. So, in many cases, the truncation error (i.e., the L^2 norm of the difference between f_0 and $f_0^{(K)}$) is negligible even when K is small (e.g., $K = 5$). Refer to Appendices A.2 and A.3 for a formal statement of the assumptions and a more detailed discussion of the truncation error, respectively.

3.2 Empirical findings

We design a simulation study to investigate the effectiveness of our proposed method in estimating $\boldsymbol{\beta}$. The parameters are as follows. The synthetic data consists of $n = 500$ observations. We once again vary the number of covariates (i.e., $p \in \{20, 100, 400\}$) to see how it affects estimation performance in high-dimensional settings. The number of terms in the Hermite expansion is $K = 5$. The true parameters are $\boldsymbol{\beta} = (1.5, -2, -0.5, 1.75, 0.25)^\top$. The main loop runs for 10 iterations and tests three estimation methods: (1) standard OLS, (2) DML with lasso, and (3) DML with RF. DML is implemented with 5-fold cross-fitting to prevent overfitting. We once again report the mean estimates $\hat{\boldsymbol{\beta}}$, bias, RMSE, and the empirical coverage probability of the 95% CI. The code, which contains all simulation details, is found in Appendix C.2.

OLS	$\hat{\boldsymbol{\beta}}$	0.725	-2.269	-0.222	1.710	0.344
	Bias	-0.775	-0.269	0.278	-0.040	0.094
	RMSE	7.724	1.195	1.026	0.838	0.266
DML (lasso)	$\hat{\boldsymbol{\beta}}$	1.168	-2.133	-0.554	1.561	0.258
	Bias	-0.332	-0.133	-0.054	-0.189	0.008
	RMSE	2.375	0.548	0.261	0.422	0.094
DML (RF)	$\hat{\boldsymbol{\beta}}$	1.799	-1.926	-0.433	1.721	0.268
	Bias	0.299	0.074	0.067	-0.029	0.018
	RMSE	1.791	0.678	0.491	0.394	0.107

Table 2: Simulation study results for $p = 400$. The entries in the $\hat{\boldsymbol{\beta}}$ rows are ordered by index (i.e., the first entry corresponds to $\hat{\beta}_1$). The method yielding the smallest average RMSE is highlighted.

Table 2 shows $\hat{\boldsymbol{\beta}}$, bias, and RMSE for $p = 400$. Table 3 shows the results for $p = 20$ and $p = 100$ and includes the coverage probability. For each value of p, we observe that DML with RF generally yields the best results. This suggests that the setting where $f_0(D)$ is nonlinear is particularly well suited for more complex ML models. We also observe that larger p tends to yield higher bias and RMSE. This experiment also demonstrates that lasso may be insufficient to handle nonlinear $f_0(D)$. One may also consider using neural networks (NN) in the DML framework because they are able to learn complex, nonlinear structures.

3.3 Extension to multivariate treatments

Suppose that we have data $\boldsymbol{X} = (X_1, ..., X_n)^\top$ with treatment $\boldsymbol{D} = (D_1, ..., D_n)^\top$. Our discussion thus far has focused on the case where each $D_i \sim \mathcal{N}(0, 1)$. Consider the setting where $D_i \in \mathbb{R}^m$ (i.e., we now estimate the joint non-linear effects of m treatments simultaneously). If each D_i is sampled i.i.d. from a multivariate standard normal, then our Hermite expansion approach remains valid. However, what if instead each D_i is sampled i.i.d. from an anisotropic Gaussian (i.e., $\mathcal{N}(0, \Sigma)$)? This motivates extending Hermite polynomials to this setting. Let $\tau \in \mathbb{R}$ denote the standard deviation of the Gaussian weight function ω. For $x \sim \mathcal{N}(0, \tau^2)$,

$$\mathrm{He}_k^\tau(x) = (-1)^k \tau^{2k} e^{\frac{x^2}{2\tau^2}} \left(\frac{d^k}{dx^k} e^{-\frac{x^2}{2\tau^2}} \right)$$

are *generalized* Hermite polynomials. With more effort, one may also derive the corresponding coefficients. Extending the DML framework to this setting certainly warrants further investigation, and we will leave that for future work. A more detailed discussion of this approach is contained in Appendix B.

4 Conclusion

In this report, we discussed the double machine learning (DML) framework for estimating treatment effects in high-dimensional settings. We proposed an extension to estimating nonlinear treatment effect functions that can be approximated by a finite Hermite expansion. Our simulations show that DML generally outperforms ordinary least squares. Future directions include using neural networks in the nonlinear setting and adapting the Hermite expansion approach to multivariate treatment effects.

References

[1] John P Boyd. "Asymptotic coefficients of hermite function series". In: *Journal of Computational Physics* 54.3 (1984), pp. 382–410. ISSN: 0021-9991. DOI: https://doi.org/10.1016/0021-9991(84)90124-4. URL: https://www.sciencedirect.com/science/article/pii/0021999184901244.

[2] Victor Chernozhukov et al. *Double/debiased machine learning for treatment and causal parameters*. 2024. arXiv: 1608.00060 [stat.ML]. URL: https://arxiv.org/abs/1608.00060.

[3] P. M. Robinson. "Root-n-consistent semiparametric regression". In: *Econometrica* 56.4 (1988), pp. 931–954. ISSN: 00129682, 14680262. URL: http://www.jstor.org/stable/1912705 (visited on 03/13/2026).

[4] Gábor Szegő. *Orthogonal Polynomials*. 4th ed. Colloquium publications - American Mathematical Society ; v. 23. Providence: American Mathematical Society, 1975. ISBN: 0821810235.

[5] Haiyong Wang and Lun Zhang. *Convergence analysis of Hermite approximations for analytic functions*. 2025. arXiv: 2312.07940 [math.NA]. URL: https://arxiv.org/abs/2312.07940.

A Finite Hermite expansion of the treatment effect function

A.1 Hermite polynomials

For reference, the first eleven probabilist's Hermite polynomials are:

$$\begin{aligned}
\mathrm{He}_0(x) &= 1,\\
\mathrm{He}_1(x) &= x,\\
\mathrm{He}_2(x) &= x^2 - 1,\\
\mathrm{He}_3(x) &= x^3 - 3x,\\
\mathrm{He}_4(x) &= x^4 - 6x^2 + 3,\\
\mathrm{He}_5(x) &= x^5 - 10x^3 + 15x,\\
\mathrm{He}_6(x) &= x^6 - 15x^4 + 45x^2 - 15,\\
\mathrm{He}_7(x) &= x^7 - 21x^5 + 105x^3 - 105x,\\
\mathrm{He}_8(x) &= x^8 - 28x^6 + 210x^4 - 420x^2 + 105,\\
\mathrm{He}_9(x) &= x^9 - 36x^7 + 378x^5 - 1260x^3 + 945x,\\
\mathrm{He}_{10}(x) &= x^{10} - 45x^8 + 630x^6 - 3150x^4 + 4725x^2 - 945.
\end{aligned}$$

A.2 Setup and assumptions

Consider the PLR model with a nonlinear treatment effect function for n observations, i.e.,

$$\boldsymbol{Y} = f_0(\boldsymbol{D})\theta_0 + g_0(\boldsymbol{X}) + \boldsymbol{\varepsilon}.$$

Here, $\boldsymbol{Y} \in \mathbb{R}^n$, $\boldsymbol{D} \in \mathbb{R}^n$, $f_0 : \mathbb{R}^n \to \mathbb{R}^n$, $\boldsymbol{X} \in \mathbb{R}^{n\times p}$, $g_0 : \mathbb{R}^{n\times p} \to \mathbb{R}^n$, $\boldsymbol{\varepsilon} \in \mathbb{R}^n$. We have the following technical conditions that are roughly stated in Section 3.

Assumption A.1. *Let $\boldsymbol{D} = (D_1, ..., D_n)^\top \in \mathbb{R}^n$. The elements of $\boldsymbol{D}$ are sampled i.i.d. from $\mathcal{N}(0, 1)$.*

Assumption A.2. *The treatment effect function $f_0(x)$ is analytic and satisfies*

$$\int_{-\infty}^{\infty} |f_0(x)|^2 e^{-x^2/2} dx < \infty,$$

and thus admits an infinite Hermite expansion

$$f_0(x) = \sum_{k=0}^{\infty} c_k He_k(x), \quad c_k = \frac{1}{k!}\int_{-\infty}^{\infty} f(x) He_k(x) \frac{e^{-x^2/2}}{\sqrt{2\pi}} dx.$$

A.3 Truncation error

The following result quantifies the error between f_0 and $f_0^{(K)}$.

Lemma A.1. *Let ω be the Gaussian weight function. The truncation error is*

$$\left\| f_0 - f_0^{(K)} \right\|_\omega^2 = \sum_{k=K}^{\infty} \frac{c_k^2}{k!}.$$

Proof. Let $f_0(x) = \sum_{k=0}^{\infty} a_k \mathrm{He}_k(x)$ and $f_0^{(K)}(x) = \sum_{k=0}^{K} a_k \mathrm{He}_k(x)$, where $a_k = \frac{c_k}{k!}$. Let $\omega(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$. The standard orthogonality relation for probabilist's Hermite polynomials is

$$\int_{-\infty}^{\infty} \mathrm{He}_m(x)\mathrm{He}_n(x)\omega(x)dx = n!\delta_{mn}.$$

The truncation error is simply

$$\begin{aligned} f_0(x) - f_0^{(K)}(x) &= \sum_{k=K}^{\infty} a_k \mathrm{He}_k(x) \\ \implies \left\| f_0(x) - f_0^{(K)}(x) \right\|_w^2 &= \int_{-\infty}^{\infty} \left| f_0 - f_0^{(K)} \right|^2 \frac{e^{-x^2/2}}{\sqrt{2\pi}} dx \\ &= \sum_{k=K}^{\infty} \frac{c_k^2}{k!} \end{aligned}$$

by orthogonality and an application of Parseval's identity. □

For many "nice" functions the Hermite coefficients decay exponentially, which implies that the truncation error also decays (near-)exponentially. In particular, if f_0 is an entire function of exponential type, the coefficients c_k decay faster than factorials, thus leading to super-exponential convergence. This means that having only a few terms in the expansion still yields a good approximation. Extending our analysis to functions that are more pathological is left for future work. For a recent work on the convergence of Hermite approximations, see [5].

A.4 Results

p	Method					
20	**OLS**					
	$\hat{\boldsymbol{\beta}}$	1.332	-2.432	-0.301	1.903	0.270
	Bias	-0.168	-0.432	0.199	0.153	0.020
	RMSE	3.326	1.427	0.699	0.352	0.159
	Coverage	1.000	1.000	1.000	1.000	1.000
	DML (lasso)					
	$\hat{\boldsymbol{\beta}}$	1.544	-2.378	-0.520	1.932	0.225
	Bias	0.044	-0.378	-0.020	0.182	-0.025
	RMSE	3.379	1.046	0.307	0.523	0.146
	Coverage	1.000	0.900	1.000	1.000	1.000
	DML (RF)					
	$\hat{\boldsymbol{\beta}}$	1.946	-1.987	-0.404	1.908	0.217
	Bias	0.446	0.013	0.096	0.158	-0.033
	RMSE	2.411	0.457	0.338	0.477	0.152
	Coverage	1.000	1.000	1.000	1.000	1.000
100	**OLS**					
	$\hat{\boldsymbol{\beta}}$	2.125	-2.026	-0.488	1.664	0.266
	Bias	0.625	-0.026	0.012	-0.086	0.016
	RMSE	2.997	0.945	1.077	0.285	0.278
	Coverage	0.900	0.900	0.900	0.900	0.700
	DML (lasso)					
	$\hat{\boldsymbol{\beta}}$	2.400	-1.955	-0.721	1.739	0.183
	Bias	0.900	0.045	-0.221	-0.011	-0.067
	RMSE	3.053	0.835	0.814	0.255	0.193
	Coverage	0.900	0.900	0.900	0.900	0.800
	DML (RF)					
	$\hat{\boldsymbol{\beta}}$	1.734	-1.960	-0.605	1.696	0.246
	Bias	0.234	0.040	-0.105	-0.054	-0.004
	RMSE	1.178	0.789	0.654	0.247	0.142
	Coverage	1.000	0.900	0.800	1.000	1.000
400	**OLS**					
	$\hat{\boldsymbol{\beta}}$	0.725	-2.269	-0.222	1.710	0.344
	Bias	-0.775	-0.269	0.278	-0.040	0.094
	RMSE	7.724	1.195	1.026	0.838	0.266
	Coverage	0.900	1.000	0.900	1.000	1.000
	DML (lasso)					
	$\hat{\boldsymbol{\beta}}$	1.168	-2.133	-0.554	1.561	0.258
	Bias	-0.332	-0.133	-0.054	-0.189	0.008
	RMSE	2.375	0.548	0.261	0.422	0.094
	Coverage	1.000	1.000	1.000	0.900	1.000
	DML (RF)					
	$\hat{\boldsymbol{\beta}}$	1.799	-1.926	-0.433	1.721	0.268
	Bias	0.299	0.074	0.067	-0.029	0.018
	RMSE	1.791	0.678	0.491	0.394	0.107
	Coverage	1.000	1.000	1.000	1.000	1.000

Table 3: Simulation study results. The entries in the $\hat{\boldsymbol{\beta}}$ rows are ordered by index (i.e., the first entry corresponds to $\hat{\beta}_1$). The method yielding the smallest average RMSE is highlighted.

B Generalized Hermite polynomials

Consider the setting where each treatment D_i is a vector in $\mathbb{R}^n$ and is sampled i.i.d. from an anisotropic Gaussian $\mathcal{N}(0, \Sigma)$, where $\Sigma \neq I$. Our Hermite expansion approach would fail here because we lose orthogonality. Moreover, the treatments are correlated with each other. This setting is relevant because it reflects many real-world multivariate treatments. Consider the following example in environmental economics. Suppose D_i represents exposure to multiple air pollutants. These are often generated by the same processes, so their distributions are highly correlated and have different variances. There are different ways of dealing with this issue. One is simply decorrelating the treatments: we use the Cholesky decomposition or principal component analysis to rotate and scale $\boldsymbol{D} = (D_1, ..., D_n)^\top$ into an isotropic, uncorrelated space. Alternatively, we can extend Hermite polynomials to accommodate general Gaussians, which will also preserve some desirable properties and allow for a more robust theoretical analysis down the line. This motivates the development of a class of Hermite polynomials that are orthogonal with respect to weight functions corresponding to Gaussians with different variances.

Definition B.1. *Given* $\tau \in \mathbb{R}$*,*

$$He_k^\tau(x) = (-1)^k \tau^{2k} e^{\frac{x^2}{2\tau^2}} \left(\frac{d^k}{dx^k} e^{-\frac{x^2}{2\tau^2}} \right)$$

are ***generalized Hermite polynomials****.*

Here, τ can be seen as the standard deviation of the Gaussian weight function. One can also derive the generalized Hermite coefficients. In order for these polynomials to be useful, we will have to prove various results that are analogous to the properties of the standard case (e.g., orthogonality, generating functions). The idea of extending Hermite polynomials to general Gaussians is well established [4], and we have done some preliminary work on them (which we will formalize at a later time). We expect these polynomials to be of independent interest and to have broader applications.

C Code

C.1 Simulation study: Comparing DML and OLS

```
library(DoubleML)
library(mlr3)
library(mlr3learners)
library(mlr3pipelines)
library(dplyr)
library(ggplot2)
library(latex2exp)

# Data generation
set.seed(168)
n_rep = 1000
n = 500
p = 20 # Repeat for p = 100 and p = 400
alpha = 2
data = list()

for (i_rep in seq_len(n_rep)) {
  data[[i_rep]] = make_plr_CCDDHNR2018(alpha = alpha,
                                       n_obs = n,
                                       dim_x = p,
                                       return_type = "data.frame")
}

# Learner setup
```

```
learner_lasso = lrn("regr.cv_glmnet", s = "lambda.min")
learner_rf = lrn("regr.ranger", num.trees = 500, max.depth = 5)

results_df = data.frame(
  Replication = integer(),
  Method = character(),
  Estimate = numeric(),
  StdError = numeric()
)

# Main loop
n_run = 10

for (i in 1:n_run) {
  df = data[[i]]

  # OLS
  ols_model = lm(y ~ ., data = df)
  ols_est = coef(summary(ols_model))["d", "Estimate"]
  ols_se = coef(summary(ols_model))["d", "Std. Error"]

  results_df = rbind(results_df, data.frame(
    Replication = i,
    Method = "OLS",
    Estimate = ols_est,
    StdError = ols_se
  ))

  dml_data = DoubleMLData$new(df, y_col = "y", d_cols = "d")

  # DML (lasso)
  dml_lasso = DoubleMLPLR$new(dml_data,
                              ml_l = learner_lasso,
                              ml_m = learner_lasso,
                              n_folds = 5)
  dml_lasso$fit()
  results_df = rbind(results_df, data.frame(
    Replication = i,
    Method = "DML_Lasso",
    Estimate = dml_lasso$coef,
    StdError = dml_lasso$se
  ))

  # DML (RF)
  dml_rf = DoubleMLPLR$new(dml_data,
                           ml_l = learner_rf,
                           ml_m = learner_rf,
                           n_folds = 5)
  dml_rf$fit()
  results_df = rbind(results_df, data.frame(
    Replication = i,
    Method = "DML_RF",
    Estimate = dml_rf$coef,
    StdError = dml_rf$se
  ))
}

# Results
theta_true = 2
```

```
    performance_metrics = results_df %>%
      group_by(Method) %>%
      summarize(
        Mean_Estimate = mean(Estimate),
        Bias = mean(Estimate) - theta_true,
        RMSE = sqrt(mean((Estimate - theta_true)^2)),
        Coverage_95 = mean((Estimate - 1.96 * StdError <= theta_true) &
                           (Estimate + 1.96 * StdError >= theta_true))
      ) %>%
      arrange(RMSE)

    print(performance_metrics)
```

C.2 Simulation study: Estimating nonlinear treatment effect functions

```
library(orthopolynom)
library(randomForest)
library(mvtnorm)
library(glmnet)

set.seed(168)

# Parameters and setup
n_sims = 10
n = 500
p = 400
K = 5
true_betas = c(1.5, -2, -0.5, 1.75, 0.25)

est_ols = est_lasso = est_rf = matrix(0, nrow=n_sims, ncol=K)
se_ols  = se_lasso  = se_rf  = matrix(0, nrow=n_sims, ncol=K)

# Helper function for cross-fitting
run_dml = function(Y, H, X, method) {
  folds = sample(rep(1:5, length.out = length(Y)))
  res_Y = numeric(length(Y))
  res_H = matrix(0, length(Y), K)

  for(f in 1:5) {
    train = which(folds != f)
    test  = which(folds == f)

    if(method == "lasso") {
      cv_Y = cv.glmnet(X[train,], Y[train], alpha=1)
      res_Y[test] = Y[test] - predict(cv_Y, newx=X[test,], s="lambda.min")

      for(k in 1:K) {
        cv_H = cv.glmnet(X[train,], H[train,k], alpha=1)
        res_H[test, k] = H[test,k] - predict(cv_H, newx=X[test,], s="lambda.min")
      }
    } else {
      rf_Y = randomForest(X[train,], Y[train])
      res_Y[test] = Y[test] - predict(rf_Y, newdata=X[test,])

      for(k in 1:K) {
        rf_H = randomForest(X[train,], H[train,k])
        res_H[test, k] = H[test,k] - predict(rf_H, newdata=X[test,])
```

```
      }
    }
  }

  dml_mod = lm(res_Y ~ res_H - 1)
  return(list(est = coef(dml_mod), se = summary(dml_mod)$coefficients[,2]))
}

# Main loop
for(i in 1:n_sims) {

  # Generate data
  X = matrix(rnorm(n * p), n, p)

  D = X[,1] + 0.5*X[,2] - 3*X[,3] + rnorm(n)
  D_std = scale(D)[,1]

  # Generate probabilist's Hermite basis
  he_polys = hermite.he.polynomials(K)
  H_list = polynomial.values(he_polys, D_std)
  H_mat = do.call(cbind, H_list)[, -1]
  colnames(H_mat) = paste0("He", 1:K)

  # True nonlinear effect and nonlinear nuisance
  f0_D = H_mat %*% true_betas
  g0_X = sin(X[,1]) + X[,5]^2 - X[,10]^5 + exp(X[,12]) - cos(X[,15])
  Y = as.numeric(f0_D + g0_X + rnorm(n))

  # OLS
  df = data.frame(Y = Y, H_mat, X)
  ols_mod = lm(Y ~ ., data = df)
  est_ols[i, ] = coef(ols_mod)[2:(K+1)]
  se_ols[i, ]  = summary(ols_mod)$coefficients[2:(K+1), 2]

  # DML (lasso)
  dml_lasso = run_dml(Y, H_mat, X, method="lasso")
  est_lasso[i, ] = dml_lasso$est
  se_lasso[i, ]  = dml_lasso$se

  # DML (RF)
  dml_rf = run_dml(Y, H_mat, X, method="rf")
  est_rf[i, ] = dml_rf$est
  se_rf[i, ]  = dml_rf$se
}

# Compute results
calc_metrics = function(est, se, truth) {
  avg_est = colMeans(est)
  bias = avg_est - truth
  rmse = sqrt(colMeans(sweep(est, 2, truth)^2))

  lower = est - 1.96 * se
  upper = est + 1.96 * se
  coverage = colMeans(sweep(lower, 2, truth, "<=") & sweep(upper, 2, truth, ">="))

  res = rbind(Average_Estimate = avg_est,
              Bias = bias,
              RMSE = rmse,
              Coverage = coverage)
```

```
  return(round(res, 3))
}

print(matrix(true_betas, nrow=1, dimnames=list("Truth", paste0("He", 1:K))))
print(calc_metrics(est_ols, se_ols, true_betas))
print(calc_metrics(est_lasso, se_lasso, true_betas))
print(calc_metrics(est_rf, se_rf, true_betas))
```